

Plan — Refonte & perfectionnement du module Absences

> Destination finale : ce plan sera copié dans `docs/abs.md` après sortie du mode Plan.

> Décisions utilisateur : > - Refonte **complète** (chantiers A → E + bonus workflow). > - Mode salle **binaire pur** : liste verticale d'étudiants ; par défaut **Présent**, un tap bascule en **Absent** (2^e tap re-bascule). Uniquement 2 statuts. Les statuts Sanctionné / Justifié sont traités après coup par le DA via le mode bureau. Pas de scan QR ni d'offline queue dans cette itération. > - Workflow validation des justificatifs (DA / responsable filière) inclus dans le périmètre.

1. Contexte

Le module `/dashboard/absences/*` couvre 8 écrans (hub, saisir, importer, etudiant, rapport, stats, fiches, portail) et le backend Django expose 3 modèles (`Etudiant` , `Presence` , `SeuilAbsence`) + 5 endpoints clés (`bulk` , `upload-justificatif` , `rapport` , `par-etudiant` , `importer`). Trois problèmes bloquants ont été identifiés :

1. **Pièces justificatives non uploadées** dans `saisir/` — le fichier choisi est gardé en mémoire locale, le nom est seulement concaténé dans le `commentaire` . Le endpoint

`/api/v1/absences/presences/upload-justificatif/` n'est **jamais** appelé depuis la saisie. 2.

Rapports partiellement fonctionnels — les filtres `dateDebut` / `dateFin` et le mode "période" sur `rapport/` ne sont pas transmis au backend ; les stats globales de `stats/` n'exposent aucun filtre département ; le donut genre affiche des zéros. 3. **Expérience mobile pauvre** — la matrice de `saisir/` utilise `overflow-x-auto` sans densification adaptée. Avec des agents pédagogiques qui pointeront sur tablette/mobile en salle pendant les séances planifiées, l'écran actuel est inutilisable (colonnes < 80 px, actions cachées au hover).

En plus de ces 3 chantiers, des inconsistances transverses doivent être corrigées : absence de contrôles de rôle sur 7 des 8 pages, aucune modale justificatif unifiée, `alert()` / modale slate sur le portail, pas de pagination sur `etudiant/` et `fiches/` , save bulk non atomique (1 POST par séance → N requêtes série).

Objectif : transformer le module en outil terrain, sécurisé par rôle, avec vrais justificatifs, rapports filtrables et UI mobile-first pour la saisie en séance.

2. État des lieux (résumé)

Backend /api/v1/absences/

- `EtudiantViewSet` : CRUD + `importer()` (Excel). - `PresenceViewSet` : CRUD + `bulk()`, `upload-justificatif()`, `rapport()`, `par-etudiant()`. - `SeuilAbsenceView` : singleton (pk=1), RBAC admin/DA. - Statut presence : 0=Present, 1=Absent, 2=Sanctionne, 3=Justiffee. - `justificatif` = `FileField upload_to='justificatifs/'`.

Frontend existant (problèmes)

Primitives réutilisables (déjà dans le repo)

- `lib/api.ts` : `apiFetch`, `apiUpload (progress)`, `apiFetchPaginated`, `apiFetchBlob`. - `lib/auth.ts` : `canAccess('absences','modifier','supprimer')`, `getStoredUser`, `rôles ( admin , DA , enseignant , responsable_filiere , etudiant ...)`. - `components/ui/` : `DataTable`, `Badge`, `StatusPill`, `FileDropzone`, `ConfirmModal`, `Pagination`, `EmptyState`, `LoadingSkeleton`, `Drawer`, `FormField`. - `components/ui/Toast.tsx` → `useToast()` (remplace `lib/flash.ts legacy`). - `chart.js` + `react-chartjs-2` + `chartjs-plugin-datalabels`. - `lib/downloadBlob.ts`, `xlsx` (déjà dépendance).

3. Stratégie proposée (6 chantiers)

Chantier A — Couche API + types absences (fondation)

Centraliser tous les appels dans un nouveau `lib/api/absences.ts` (même pattern que `lib/api/scolarite.ts`), et poser les types dans `types/absences.ts`.

Nouveau module : - `etudiants.list(params)`, `.get(id)`, `.import(file, onProgress)`. - `presences.list(params)`, `.bulk(payload)`, `.uploadJustificatif(presenceId, file, onProgress)`, `.rapport(params)`, `.parEtudiant(etudiantId, params)`. - `seuil.get()`, `seuil.update(value)`. - Types : `Etudiant`, `Presence`, `StatutPresence (enum)`, `RapportRow`, `SeuilAbsence`.

Pattern cache TTL (voir `lib/cache.ts`) pour : liste départements, semaines générées, étudiants d'un département (TTL 5 min).

Chantier B — Composants partagés absences

Créer 5 composants dans `components/absences/` :

1. `StatutBadge.tsx` — encapsule le mapping statut → couleur/label (remplace 8 implémentations manuelles).
2. `JustificatifUploader.tsx` — wrapper autour de `FileDropzone` + `apiUpload` vers `/presences/:id/upload-justificatif/`, barre de progression, preview + téléchargement du justificatif existant. Utilisé par `saisir/`, `portail/`, `etudiant/`.
3. `AbsenceStatutPicker.tsx` — sélecteur cyclique $P \rightarrow A \rightarrow S \rightarrow J$ avec tap-targets ≥ 44 px (mobile-friendly, gestes tactiles).
4. `EtudiantAutocomplete.tsx` — recherche debouncée (300 ms) avec restriction par rôle (un enseignant ne voit que les étudiants de ses séances, un étudiant ne se voit que lui-même).
5. `PeriodeFilter.tsx` — filtre unifié (presets 7/30/60/90j + custom + par mois/semestre), réutilisé par `rapport/`, `stats/`, `etudiant/`.

Chantier C — Refonte `saisir/` en 2 modes

Règle de rôle transverse : les actions autorisées dépendent du rôle. - `enseignant` / agent terrain → **Présent / Absent** uniquement (statuts 0 et 1) + upload justificatif. - `DA` / `responsable_filiere` / `admin` → tous les statuts modifiables, requalification Sanctionné/Justifié depuis l'écran d'arbitrage (chantier F).

L'écran devient adaptatif **selon la taille de viewport + rôle** :

Mode Bureau ($\geq 1g$) : matrice actuelle améliorée. - Toujours-visibles : boutons commentaire + justificatif (plus cachés au hover → tactile-compatible). - Cycle de statut (clic cellule) : - Rôles agent/enseignant → cycle **Présent** ↔ **Absent** uniquement. - Rôles DA/resp.filière/admin → cycle complet **P** → **A** → **S** → **J** → **P**. - Autosave debounced 3 s (ou bouton manuel conservé). - Bulk save optimisé : **1 seul POST** groupé par requête (`Promise.all` avec `Promise.allSettled` pour tolérer les erreurs partielles). - Colonne "total" sticky à droite. - Barre de progression globale (X/Y séances sauvegardées).

Mode Salle (`< md` OU URL `?mode=salle`) : interface simple pensée pour un agent qui pointe debout en salle, téléphone en main.

1. **Écran 1 — Sélection séance** : liste des séances du jour (pré-filtrée sur jour courant) ; tap sur une ligne → écran 2. Pré-sélection intelligente si l'heure courante tombe dans un créneau.
2. **Écran 2 — Pointage 1-tap binaire** :
 - Header collant : nom du prof / matière / créneau / salle.
 - Compteur en temps réel en haut : **X Présents / Y Absents**.
 - **Liste verticale des étudiants** (matricule + nom, card ≥ 64 px, pleine largeur).
 - **État par défaut : tous Présents** (card fond vert clair + icône ✓).
 - **Un tap sur la card** → bascule en **Absent** (fond rouge + icône X). Un 2^e tap → retour **Présent**. Un seul bouton = toute la card est tappable. Pas de menu, pas de sheet.
 - **Icône** 📷 apparaît à droite de la card dès qu'elle est en "Absent", pour joindre un justificatif photo/PDF via `Choisir un fichier` `Aucun fichier choisi` (caméra native sur mobile).
 - Recherche par matricule en haut (utile pour liste > 30 étudiants).
3. **Écran 3 — Validation** : récap

"X présents / Y absents" + bouton plein écran "Valider la séance" → POST `presences.bulk` (statuts 0=Présent, 1=Absent) + upload des justificatifs attachés en séquentiel → toast succès → retour écran 1.

Pourquoi binaire : les statuts "Sanctionné" et "Justifié" sont des qualifications post-hoc qui dépendent d'un arbitrage administratif (commentaire étudiant, document fourni, décision DA). Les exposer à l'agent en salle créerait du bruit et des saisies incohérentes. Ces statuts restent modifiables via le mode bureau et le chantier F (validation justificatifs).

Les deux modes (bureau et salle) partagent le même hook `useSaisieAbsences` pour la cohérence des données et de la logique métier.

Chantier D — Rapports, stats & exports fonctionnels

`rapport/page.tsx` : - Transmettre `date_debut`, `date_fin`, `mois`, `departement`, `niveau`, `filiere` au backend (vérifier que `PresenceViewSet.rapport()` accepte ces params — si non, liste des `queryparams` à ajouter côté backend dans la section "Dépendances backend"). - Ajouter export Excel via `xlsx` (déjà installé) et export PDF via print CSS dédié. - Remplacer KPI actuels par cartes `StatutBadge` -colorées + mini-sparkline par étudiant (évolution sur période).

`stats/page.tsx` : - Filtre département ajouté. - Supprimer le donut genre (données non exploitées) → remplacer par **pyramide d'absentéisme par jour de la semaine** (utile pour détecter jours à risque). - Seuil dans une carte dédiée avec `ConfirmModal` avant PATCH. - Liste rouge : limiter à top 20, lien "voir tous" → page dédiée paginée. - Ajout d'une courbe temporelle (absences/jour sur la période sélectionnée).

`etudiant/page.tsx` : - Filtre dates transmis en queryparam (`date_debut`, `date_fin`). - Pagination sur l'historique (`apiFetchPaginated`). - Protection RBAC : un `etudiant` ne peut chercher que lui-même (redirection auto vers `portail/absences`).

Chantier E — Portail étudiant + réclamations

- Unifier la modale réclamation sur `ConfirmModal` + `FileDropzone` + `JustificatifUploader` (cohérence design system). - Afficher badge + historique des réclamations (accepted / en_examen / refusée) sur chaque ligne. - Toast `useToast()` après soumission (pas juste fermeture silencieuse). - Ajouter un compteur "X absences au-dessus du seuil → contacter votre DA" en tête de page si applicable.

Chantier F — Arbitrage justificatifs & sanctions (DA / responsable_filiere)

Séparation des rôles validée : - **Agent en salle** → peut uniquement marquer **Présent / Absent** (chantier C mode salle). - **Prof / enseignant** → mode bureau (4 statuts visibles mais seuls Présent/Absent modifiables). - **DA / responsable_filiere / admin** → **seuls habilités** à requalifier une absence en **Sanctionné (2)** ou **Justifiée (3)** via le nouvel écran dédié.

Nouvel écran `/dashboard/absences/justificatifs/` accessible aux rôles `DA`, `responsable_filiere`, `admin` : - **Onglet 1 — Justificatifs en attente** : toutes les `Presence` avec `justificatif` uploadé et `statut = 1` (Absent). - Colonnes : matricule, nom, date séance, matière, aperçu justificatif (preview PDF/image via `JustificatifPreview`), commentaire étudiant, actions. - Actions : **Valider** (→ `statut = 3` Justifiée) / **Refuser** (→ reste `statut = 1`, ajoute commentaire de refus). - **Onglet 2 — Sanctions** : le DA peut requalifier une absence en `statut = 2` (Sanctionné) depuis la liste filtrée des absents. - Action : **Sanctionner** (→ `statut = 2`, commentaire obligatoire justifiant la sanction). - **Filtres** : département, période, type de statut. - **Pagination** + tri par date. - **Compteur global** "X justificatifs en attente" affiché comme badge sur l'entrée de menu + sur le hub `/dashboard/absences`. - **Journal d'audit** : chaque changement de statut loggue `user_id`, `ancien_statut`, `nouveau_statut`, `commentaire`, `date` (champ déjà présent via `date_modification` ; ajouter un log séparé si besoin d'historique complet → ticket backend si applicable).

Nécessite côté backend : - Endpoint `POST /api/v1/absences/presences/:id/changer-statut/` prenant `{statut, commentaire}` et vérifiant le rôle (`DA admin`). - Ou extension de l'endpoint existant `bulk()` pour accepter un changement de statut en masse avec garde RBAC. - Cf. § 5 pour coordination backend.

4. Fichiers à modifier / créer

À créer

- `lib/api/absences.ts` — couche API. - `types/absences.ts` — types TS. -
- `components/absences/StatutBadge.tsx` — mapping des 4 statuts backend (P/A/S/J). -
- `components/absences/JustificatifUploader.tsx` — upload réel vers `/upload-justificatif/`. -
- `components/absences/AbsenceStatutPicker.tsx` — pour mode bureau : cycle conditionnel par rôle (binaire pour agent, 4 statuts pour DA). - `components/absences/PresentAbsentToggle.tsx` — pour mode salle : bouton binaire 1-tap (Présent ↔ Absent). -
- `components/absences/EtudiantAutocomplete.tsx` - `components/absences/PeriodeFilter.tsx` -
- `components/absences/JustificatifPreview.tsx` — aperçu inline PDF/image pour l'écran d'arbitrage (chantier F). - `hooks/useSaisieAbsences.ts` — logique partagée saisir bureau/salle, gère les permissions par rôle. - `app/dashboard/absences/saisir/salle/page.tsx` — mode salle : liste des

séances du jour. - `app/dashboard/absences/saisir/salle/[suiviId]/page.tsx` — pointage binaire d'une séance. - `app/dashboard/absences/justificatifs/page.tsx` — arbitrage DA (justifications + sanctions).

À modifier

- `app/dashboard/absences/page.tsx` — corriger liste rouge, ajouter lien "mode salle". - `app/dashboard/absences/saisir/page.tsx` — refonte mode bureau + câblage vraie upload justificatif.
- `app/dashboard/absences/importer/page.tsx` — template downloadable, validation client. - `app/dashboard/absences/etudiant/page.tsx` — RBAC, pagination, filtre dates serveur. - `app/dashboard/absences/rapport/page.tsx` — filtres fonctionnels, exports Excel/PDF. - `app/dashboard/absences/stats/page.tsx` — filtres, refonte donut, courbe temporelle. - `app/dashboard/absences/fiches/page.tsx` — garde-fou volume + pagination fiches. - `app/dashboard/portail/absences/page.tsx` — cohérence design system, historique réclamations.

Primitives réutilisées (sans modification)

- `lib/api.ts`, `lib/auth.ts`, `lib/downloadBlob.ts`, `lib/cache.ts`. - `components/ui/*` (DataTable, FileDropzone, ConfirmModal, Pagination, Toast, EmptyState, LoadingSkeleton, Drawer, FormField, StatusPill, Badge).

5. Dépendances backend à clarifier

Vérifier côté `apps/absence/views.py` et `serializers.py` : - `PresenceViewSet.rapport()` accepte-t-il `date_debut`, `date_fin`, `mois`, `niveau`, `filiere` ? - `PresenceViewSet.par-etudiant()` accepte-t-il `date_debut`, `date_fin` + pagination ? - Endpoint `upload-justificatif()` retourne-t-il l'URL du fichier pour preview immédiat ? - (Bonus) Endpoint de validation justificatif existe-t-il ou faut-il le créer ?

Action préalable : lire `apps/absence/views.py` (worktree `C:\react_projects\GES\siga\apps\...`) avant d'implémenter chantier D. Si filtres manquants → ticket séparé côté backend.

6. Ordre d'exécution recommandé

Refonte complète en un seul passage :

1. **Chantier A** (2 j) — couche API + types, sans impact UI, débloque tout. 2. **Chantier B** (2 j) — composants partagés, testables isolément. 3. **Chantier C mode Bureau** (2 j) — correction immédiate du bug justificatifs + cycle conditionnel par rôle. 4. **Chantier C mode Salle** (2 j) — pointage binaire mobile pour agents terrain. 5. **Chantier D** (3 j) — rapports/stats fonctionnels, après clarification backend. 6. **Chantier E** (1 j) — cohérence portail étudiant. 7. **Chantier F** (2 j) — arbitrage DA (justifications + sanctions).

Total estimé : 14 jours.

7. Vérification (test end-to-end)

1. **Bug justificatifs** : lancer `npm run dev`, se connecter en enseignant, saisir absences, joindre un PDF → vérifier dans Django admin que `Presence.justificatif` est bien rempli avec le fichier (pas juste mention en commentaire). 2. **Mode salle** : ouvrir l'URL `/dashboard/absences/saisir/salle` sur mobile (DevTools responsive + vrai téléphone), sélectionner une séance, tapoter la card d'un étudiant pour le passer en Absent, vérifier le compteur en temps réel, joindre une photo via la caméra, valider → POST bulk + upload justificatifs séquentiel. 3. **Rapport filtré** : générer rapport pour un département + période 30j, comparer avec comptage manuel d'une semaine, exporter Excel → ouvrir dans LibreOffice, vérifier les totaux. 4. **Stats** : vérifier filtre département, pyramide jours, courbe temporelle. 5. **Portail étudiant** : se connecter en étudiant, soumettre réclamation avec PDF, vérifier toast + apparition dans liste. 6. **RBAC** : tester que rôle `enseignant` ne peut pas modifier le seuil, que rôle `etudiant` ne peut pas chercher un autre étudiant. 7. **Responsive** : chaque écran testé à 360 / 768 / 1024 px (Chrome DevTools device toolbar). 8. **Checklist design system** (§ 11 de `docs/skill_design.md`) : palette, typo Cairo, libellés FR, feedback via `useToast`, `npm run lint` OK.

8. Risques & mitigations